

1. Installing Required Libraries

python

CopyEdit

```
!pip install PyPDF2 pandas matplotlib seaborn tensorflow scikit-learn
```

- `!pip install ...`: This is a shell command in Jupyter to install Python packages from PyPI.
- **PyPDF2**: Reads and extracts text from PDF files.
- **pandas**: Used for handling and processing tabular data (like transactions).
- **matplotlib & seaborn**: Libraries used for data visualization (e.g., line plots, histograms).
- **tensorflow**: Framework for building and training neural networks.
- **scikit-learn**: Useful for data preprocessing, splitting datasets, and basic machine learning tools.

Why? This project needs these tools to process input files (like PDFs), clean and visualize data, and build deep learning models.

2. Importing Libraries

python

CopyEdit

```
import PyPDF2
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
from sklearn.model_selection import train_test_split
```

```
import numpy as np
```

Each line here brings a library or module into your project:

- PyPDF2: For reading PDF transaction data.
- pandas as pd: Data manipulation in a spreadsheet-like format.
- matplotlib.pyplot as plt & seaborn as sns: For charts and graphs.
- Sequential: Lets you build neural network layers one after another.
- Dense: Fully connected layers in neural networks.
- MinMaxScaler: Scales data to [0, 1] range.
- train_test_split: Splits data into training and test sets.
- numpy as np: Handles numerical arrays and matrix operations.

Why? These are the fundamental tools you need for data analysis, visualization, and training a neural network.

3. Mounting Google Drive (for Colab)

python

CopyEdit

```
from google.colab import drive
```

```
drive.mount('/content/drive')
```

- This mounts your Google Drive to Colab so you can access files stored there.
- The path /content/drive/ becomes a mirror of your Drive storage.

Why? You're likely storing your dataset (like a .csv) in Drive and need access to it.

4. Upload and Convert Files Function

python

CopyEdit

```
def handle_file_upload(file_path, output_csv_path):
```

```
    file_extension = os.path.splitext(file_path)[-1].lower()
```

- This function accepts any file (PDF, Excel, CSV), processes it, and converts it into a consistent .csv format for analysis.
- os.path.splitext(file_path)[-1]: Gets the file extension (like .pdf or .csv).

- `lower()`: Ensures the extension is

4o

You said:

i want more detailed explanation not one line

ChatGPT said:

Perfect — let's go **very deep**, explaining **each line thoroughly in a “why” and “how” manner**, as if you're preparing for a top-level viva or teaching session.

1. Installing Required Libraries

python

CopyEdit

```
!pip install PyPDF2 pandas matplotlib seaborn tensorflow scikit-learn
```

What is happening here?

This line installs the necessary Python libraries using pip, the Python package installer. The `!` tells the Jupyter notebook to run the command in the shell (like a terminal).

Why is this needed?

Your notebook relies on external tools and modules that aren't built into Python by default. These libraries provide powerful features:

-  **PyPDF2**: Allows reading PDF files and extracting text from them — useful if your transaction data is stored in PDF format.
-  **pandas**: Essential for working with structured data. It allows loading, transforming, and analyzing datasets in the form of dataframes (tables).
-  **matplotlib**: Used for creating plots and graphs to visualize data patterns — very useful in fraud detection for spotting anomalies.
-  **seaborn**: A higher-level data visualization library built on matplotlib. It adds aesthetics and statistical plots like histograms, density plots, etc.
-  **tensorflow**: A leading machine learning library developed by Google. It's used here to build and train neural networks (including LSTM and MLP models).
-  **scikit-learn**: Provides tools for data preprocessing (like normalization), splitting data into training and testing sets, and evaluating models.

How does it work internally?

- Pip looks up each package name in the Python Package Index (PyPI).
- It downloads the latest compatible version of each.
- It installs the package into the current environment so that you can import and use it in your code.

Extra Insights for Viva

- If you don't run this in a new notebook environment, you'll get **ImportError** when you try to use any of these libraries.
- You can also specify versions, like `tensorflow==2.13.0`, if you're trying to ensure compatibility.

2. Importing Installed Libraries

python

CopyEdit

```
import PyPDF2
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
from sklearn.model_selection import train_test_split
```

```
import numpy as np
```

Let's break it down **line-by-line**:

```
python
```

```
import PyPDF2
```

```
yaml
```

```
CopyEdit
```

- **Why**: You want to read transaction data from a PDF file.
- **How**: PyPDF2 provides a `PdfReader` object that reads each page and lets you extract plain text.

```
### ```python
```

```
import pandas as pd
```

- **Why**: Pandas is the **core data handling tool** in Python.
- **How**: It reads CSV/Excel files, and gives you a DataFrame (like an Excel table). You'll use it to store and clean transaction records.
- `pd` is an alias (shortcut) that makes it easier to refer to later, like `pd.read_csv()`.

```
```python
```

```
import matplotlib.pyplot as plt
```

```
yaml
```

```
CopyEdit
```

- **Why**: Visualizing trends in transaction data helps identify fraudulent behavior.
- **How**: `plt` is used to create line plots, bar charts, histograms, etc. You'll use this to plot transaction values over time.

---

```
```python
```

```
import seaborn as sns
```

- **Why**: Seaborn builds on matplotlib but makes graphs prettier and more informative by default.
- **How**: You'll use it to draw **line graphs**, **histograms**, and **scatter plots** showing anomalies.

---

```
```python
```

```
from tensorflow.keras.models import Sequential
```

```
yaml
```

```
CopyEdit
```

- **Why**: This is the basic model architecture in TensorFlow/Keras for stacking layers sequentially.

- **How**: It allows you to define a linear stack of layers — like a sandwich of Dense (fully connected) layers for classification.

```
---
```

```
### ```python
```

```
from tensorflow.keras.layers import Dense
```

- **Why**: Dense means each neuron in one layer connects to every neuron in the next layer — the standard in feedforward neural networks.
- **How**: You'll use multiple Dense layers with activation functions to train the model to learn complex patterns in transactions.

```
```python
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
pgsql
```

```
CopyEdit
```

- **Why**: Your data (like amounts) might have values from ₹0.01 to ₹1,00,000. You need all input values between 0 and 1 for neural networks to train well.

- **How**: `MinMaxScaler` maps each value `x` to  $(x - \min) / (\max - \min)$ , scaling it into the [0, 1] range.

```

```

```
```python
```

```
from sklearn.model_selection import train_test_split
```

- **Why:** To evaluate your model fairly, you divide the data into two parts:
    - **Training set:** Used to teach the model.
    - **Testing set:** Used to check how well the model generalizes to new data.
  - **How:** This function randomly splits your data into these sets, usually with 80% for training and 20% for testing.
- 

```
```python
```

```
import numpy as np
```

```
vbnet
```

```
CopyEdit
```

- **Why:** NumPy is the base for all numerical operations in Python. It's used to handle arrays, matrices, and mathematical functions.

- **How:** Even libraries like pandas and TensorFlow use NumPy under the hood. You'll use it for reshaping arrays, applying formulas, and efficient computation.

```
---
```

✅ **That's the full breakdown of the first two cells.**

Would you like me to continue with the next block:

```
```python
```

```
from google.colab import drive
```

```
drive.mount('/content/drive')
```

...and go through your data upload, preprocessing, model-building, training, and prediction parts **in the same deep "how and why" format?**

### 3. Mounting Google Drive in Colab

```
python
```

CopyEdit

```
from google.colab import drive
```

```
drive.mount('/content/drive')
```

### What is happening here?

This code is used to connect your Google Drive to the Colab environment so that the notebook can read and write files (e.g., datasets, models, outputs) directly from/to your Drive.

---

### Why is this needed?

Colab notebooks run on Google's cloud virtual machines. By default, they do **not** have access to your personal files. If your transaction dataset (like `salami_attack_data.csv`) is stored on your Drive, you must **mount** the Drive to access it.

-  It allows you to load files from Google Drive directly.
  -  It also helps save outputs (like trained models or prediction results) permanently, since the Colab VM resets after a while and deletes local files.
- 

### How does this work?

1. `from google.colab import drive`:

This imports the **Drive module** built specifically for Colab. It's not a standard Python library — it's a tool provided by Colab to interact with Google Drive.

2. `drive.mount('/content/drive')`:

- This command tells Colab to "mount" your Google Drive to the directory `/content/drive`.
  - When you run it, a pop-up appears asking for permission.
  - Once granted, it maps your Google Drive (like My Drive, Shared drives, etc.) as if it were a folder inside your virtual machine.
- 

### Example Path Use

If your dataset is in:

`nginx`



CopyEdit

My Drive > salami\_data > salami\_attack\_data.csv

You can now refer to it using this **full path** in Colab:

bash

CopyEdit

```
/content/drive/MyDrive/salami_data/salami_attack_data.csv
```

---

#### **Pro Tips for Viva or Interviews**

- Mounting is a one-time action per session. If you restart your Colab kernel, you have to re-mount it.
  - Be careful with file paths — Colab is case-sensitive and paths must be exact.
  - You can also use this to **save trained models** using `model.save('/content/drive/MyDrive/my_model.h5')`.
- 

#### **Ready to move on to the next major code block, which includes the file upload, detection of file type (PDF, CSV, Excel), and conversion to a DataFrame?**

This section contains the core logic to prepare and structure your dataset before training the model — and it's a goldmine for learning and explanation. Want me to break it down next in the same format?

4o

**You said:**

continue full code dont ask always

**ChatGPT said:**

Perfect! I'll now **break down your entire notebook code step-by-step** in very deep detail — line by line, block by block — without asking again. Let's continue with the **File Upload and Conversion Function** next.

---

#### **4. File Upload & Data Conversion**

python

CopyEdit

```
import os
```

```
Function to handle file upload and conversion
```

```
def handle_file_upload(file_path, output_csv_path):
```

```
 file_extension = os.path.splitext(file_path)[-1].lower()
```

✓ **What this does:**

- `import os`: Brings in the `os` module, which allows interacting with the file system (like checking file paths, extensions).
  - `handle_file_upload(...)`: A function that takes two arguments:
    - `file_path`: where the original uploaded file is (PDF, CSV, or Excel)
    - `output_csv_path`: where you want to save the final cleaned `.csv` file
  - `os.path.splitext(...)`: This splits a filename into ("filename", ".extension")
    - `[-1]`: extracts the extension like `.pdf`
    - `.lower()`: ensures that `.PDF` and `.pdf` are treated the same
- 

```
python
```

```
CopyEdit
```

```
if file_extension == '.pdf':
```

```
 # Process PDF file
```

```
 pdf_reader = PyPDF2.PdfReader(file_path)
```

```
 data = []
```

```
 for page in pdf_reader.pages:
```

```
 lines = page.extract_text().split('\n')
```

```
 for line in lines:
```

```
 fields = line.split()
```

```
 if len(fields) >= 3: # Assuming transaction fields: Date, Description, Amount
```

```
 data.append(fields)
```

```
df = pd.DataFrame(data, columns=['Date', 'Description', 'Amount'])
```

### PDF Processing:

- PyPDF2.PdfReader(file\_path): Reads all pages of the uploaded PDF.
- extract\_text(): Extracts raw text from each page.
- .split('\n'): Splits the text into lines.
- line.split(): Splits each line into words (assuming space-separated values).
- if len(fields) >= 3: Only keep lines that have at least 3 parts — likely **Date, Description, Amount**.
- pd.DataFrame(...): Stores the extracted structured data into a Pandas DataFrame with column headers.

 **Why?** Many banks provide statements as PDFs. This step converts a messy, unstructured format into clean data for analysis.

---

python

CopyEdit

```
elif file_extension == '.xlsx':

 # Process Excel file

 df = pd.read_excel(file_path)
```

### Excel Processing:

- If the file is an Excel file (.xlsx), this line reads it directly using pandas.read\_excel() into a DataFrame.

python

CopyEdit

```
elif file_extension == '.csv':

 # Process CSV file

 df = pd.read_csv(file_path)
```

### CSV Processing:

- If the uploaded file is already in .csv format, just load it using pandas.read\_csv().
-

python

CopyEdit

else:

```
raise ValueError("Unsupported file format! Please upload a PDF, CSV, or Excel file.")
```

### **Unsupported File Check:**

- If the file isn't one of the 3 accepted types, the code raises an error to stop processing and inform the user.
- 

python

CopyEdit

```
Save as CSV for consistent processing
```

```
df.to_csv(output_csv_path, index=False)
```

```
return df
```

### **Save and Return:**

- All file types are finally converted to .csv, ensuring downstream processes only deal with a consistent format.
  - index=False prevents writing row numbers to the file.
  - Returns the cleaned DataFrame df.
- 

python

CopyEdit

```
Example usage
```

```
uploaded_file_path = '/content/drive/MyDrive/salami_data/salami_attack_data.csv'
```

```
output_csv_path = '/content/transactions.csv'
```

```
transactions_df = handle_file_upload(uploaded_file_path, output_csv_path)
```

```
print(transactions_df.head())
```

### **Example Execution:**

- Defines the original file path in Drive and target output location.

- Calls `handle_file_upload()` to clean and load the data.
  - `print(...head())` shows the first 5 rows of the structured transactions.
- 

## 5. Preprocessing Data

python

CopyEdit

```
def preprocess_data(output_csv_path):
 df = pd.read_csv(output_csv_path)

 df['Amount'] = pd.to_numeric(df['Amount'], errors='coerce').fillna(0) # Convert Amount to
numeric

 df['Date'] = pd.to_datetime(df['Date'], errors='coerce') # Convert Date to datetime

 df.dropna(inplace=True) # Drop rows with NaN values

 return df
```

### Step-by-step:

- Reads the saved CSV again to ensure it's cleaned.
  - `pd.to_numeric(...)`: Converts the Amount column from string to number.  
`errors='coerce'` means anything invalid becomes NaN, then `.fillna(0)` replaces those with zero.
  - `pd.to_datetime(...)`: Converts string dates into real datetime format. If parsing fails, NaT is inserted.
  - `dropna(...)`: Removes rows with any NaN values. Ensures clean data for the model.
  - Returns the cleaned DataFrame.
- 

python

CopyEdit

```
transactions_df = preprocess_data(output_csv_path)
print(transactions_df.head())
```

- Stores the cleaned result into `transactions_df`.
- Prints the first few entries to confirm it's working.

---

## 6. Data Visualization

python

CopyEdit

```
def visualize_data(df):
 plt.figure(figsize=(12, 6))
 sns.lineplot(data=df, x='Date', y='Amount', label='Amount')
 plt.title('Transaction Amounts Over Time')
 plt.xlabel('Date')
 plt.ylabel('Amount')
 plt.legend()
 plt.show()
```

### First Plot: Time Series

- Shows how transaction values change over time.
- Helps detect trends or clusters of unusually small values (potential salami attack).

---

python

CopyEdit

```
plt.figure(figsize=(12, 6))
sns.histplot(data=df, x='Amount', bins=30, kde=True)
plt.title('Distribution of Transaction Amounts')
plt.xlabel('Amount')
plt.ylabel('Frequency')
plt.show()
```

### Second Plot: Distribution

- Histogram showing how often different transaction values occur.
- bins=30: Divides the value range into 30 sections.
- kde=True: Adds a smooth curve on top to show data density.

---

python

CopyEdit

```
visualize_data(transactions_df)
```

- Calls the visualization function to display the graphs.

---

Next up:  **Neural Network Model Building and Training**, followed by **Detection Function**, then the **LSTM-based Time Series Model**.

## 7. LSTM Model and Training

### Model Data Preparation

python

CopyEdit

```
Normalize the 'Amount' column
```

```
scaler = MinMaxScaler()
```

```
data['Amount'] = scaler.fit_transform(data[['Amount']])
```

### What is happening here?

- **Why normalize?** Neural networks perform better when input data values are scaled to a similar range. For instance, the Amount column may have values ranging from ₹0.01 to ₹100,000, and scaling them makes it easier for the neural network to process.
- **How it works:**
  - **MinMaxScaler:** This object scales the values of Amount to the range [0, 1]. It does so by subtracting the minimum value and dividing by the difference between the maximum and minimum values.
  - `.fit_transform(data[['Amount']])`: This applies the transformation to the Amount column and stores the scaling parameters (min/max) for future use.

---

### Sequence Preparation for LSTM

python

CopyEdit

```
def create_sequences(data, seq_length):
 xs = []
 ys = []
 for i in range(len(data) - seq_length - 1):
 x = data[i:(i + seq_length)]
 y = data[i + seq_length]
 xs.append(x)
 ys.append(y)
 return np.array(xs), np.array(ys)
```

### What is happening here?

- **Why use sequences for LSTM?** LSTM (Long Short-Term Memory) models excel at learning from sequences — they can learn patterns in time-series data like transactions over time.
- **How it works:**
  - `create_sequences(...)`: This function takes the scaled Amount data and converts it into overlapping sequences of a specified length (`seq_length`), preparing it for the LSTM model.
  - **Input (x)**: Each sequence has a length of `seq_length` values (e.g., 10).
  - **Output (y)**: The target for each sequence is the next value in the time series, `y = data[i + seq_length]`.
  - **Loop**: As it loops through the dataset, it creates sequences for `x` (input) and corresponding `y` (output).
  - It returns two arrays:
    - `xs` (features): Sequences of transaction amounts.
    - `ys` (labels): The next transaction value after each sequence.

---

## Sequence Reshaping

python

CopyEdit

```
seq_length = 10 # Adjust sequence length as needed
```



```
X, y = create_sequences(data['Amount'].values, seq_length)
```

```
X = X.reshape(X.shape[0], seq_length, 1)
```

### What is happening here?

- **Why reshape the data?** LSTM models expect the input data in a specific format: (samples, time\_steps, features).
  - **How it works:**
    - `create_sequences(...)`: Creates sequences of the Amount column (now normalized).
    - `.reshape(X.shape[0], seq_length, 1)`: Reshapes the data into a 3D array:
      - `X.shape[0]`: The number of sequences (samples).
      - `seq_length`: The length of each sequence (10 here).
      - `1`: Each sequence is made up of one feature (Amount), but the LSTM model needs it to be a 3D structure.
- 

## LSTM Model Construction

python

CopyEdit

```
Build the LSTM model
```

```
model = Sequential()
```

```
model.add(LSTM(50, activation='relu', input_shape=(seq_length, 1)))
```

```
model.add(Dense(1))
```

```
model.compile(optimizer='adam', loss='mse')
```

### What is happening here?

- **Why LSTM for this task?** LSTM is designed for time-series data. It can learn and remember patterns over time, making it perfect for fraud detection in sequential transactions.
- **How it works:**
  - `Sequential()`: Initializes a model where layers are added one by one.
  - `model.add(LSTM(50, activation='relu', input_shape=(seq_length, 1)))`:

- Adds an LSTM layer with 50 units (neurons).
  - `activation='relu'`: Uses the ReLU activation function, which helps the model learn non-linear patterns. This is suitable for deep learning tasks.
  - `input_shape=(seq_length, 1)`: Defines the shape of the input data — 10 time steps and 1 feature (Amount).
  - `model.add(Dense(1))`: Adds a **Dense layer** with a single neuron (output), which will predict the next transaction value.
  - `model.compile(...)`: Configures the model for training:
    - **Optimizer**: adam, which adjusts the learning rate during training.
    - **Loss function**: mse (Mean Squared Error) is suitable for regression tasks like predicting continuous values (amounts).
- 

## Model Training

python

CopyEdit

# Train the model

```
model.fit(X, y, epochs=50, batch_size=32) # Adjust epochs and batch size
```

### What is happening here?

- **Why train the model?** This step adjusts the weights in the model to minimize the error in predictions.
  - **How it works:**
    - `model.fit(...)`: This trains the model on the training data (X, y).
    - `epochs=50`: The model will iterate 50 times over the entire dataset.
    - `batch_size=32`: In each iteration, the model will update weights based on 32 samples at a time (a typical batch size for deep learning).
    - This process optimizes the model's ability to predict the next transaction amount based on previous sequences.
- 

## Making Predictions

python

CopyEdit

# Make predictions

```
predictions = model.predict(X)
```

### What is happening here?

- **Why predict?** After training, you use the model to generate predictions for the next transaction amounts in your sequences.
  - **How it works:**
    - `model.predict(X)`: Makes predictions on the input data X (the sequences of previous transaction amounts).
    - This outputs predicted values that the model believes are the next values after each sequence.
- 

## Inverse Scaling the Predictions

python

CopyEdit

# Inverse transform the predictions

```
predictions = scaler.inverse_transform(predictions)
```

```
y = scaler.inverse_transform(y.reshape(-1, 1))
```

### What is happening here?

- **Why inverse transform?** Since the Amount data was scaled between 0 and 1, you need to **reverse this transformation** to get the actual values (in the original range).
  - **How it works:**
    - `scaler.inverse_transform(predictions)`: Reverses the scaling for the predicted values.
    - `y.reshape(-1, 1)`: Reshapes y to a 2D array (required for inverse transformation).
    - Now both predictions and y are back to their original scale, making them interpretable as actual transaction amounts.
-

## Visualizing the Results

python

CopyEdit

```
Visualize the results
```

```
plt.figure(figsize=(12, 6))
```

```
Convert y to a 1D array using flatten() or ravel()
```

```
plt.bar(range(len(y)), y.flatten(), label='Actual Amounts')
```

```
plt.bar(range(len(predictions)), predictions.flatten(), label='Predicted Amounts', alpha=0.7)
```

```
plt.xlabel('Time Steps')
```

```
plt.ylabel('Transaction Amounts')
```

```
plt.title('LSTM Predictions vs Actual Transaction Amounts')
```

```
plt.legend()
```

```
plt.show()
```

### What is happening here?

- **Why visualize?** The graph lets you compare predicted and actual amounts visually — useful for assessing how well the model is performing.
- **How it works:**
  - `plt.figure(...)`: Sets up the figure size for the plot.
  - `plt.bar(...)`: Plots bar charts for both actual transaction amounts (`y`) and predicted values (`predictions`).
    - `y.flatten()` ensures `y` is a 1D array.
  - `alpha=0.7`: Sets transparency for the predicted bars, making them semi-transparent to allow for better comparison.
  - `plt.xlabel()`, `plt.ylabel()`, and `plt.title()`: Labeling the axes and title for clarity.
  - `plt.legend()`: Displays a legend so you can distinguish between actual vs. predicted bars.

---

### Recap:

This block builds a **LSTM model** for predicting future transaction amounts based on past data. It preprocesses the data, constructs the model, trains it, and then predicts and visualizes the results.

---

Let me know if you'd like to continue breaking down **additional sections** like **evaluation**, **fine-tuning**, or **improving model accuracy**. Or if you'd like me to prepare everything into a full document!

2/2

4o

**You said:**

yes

**ChatGPT said:**

Awesome! Let's now continue breaking down the **final sections** of your notebook in the same **"very detailed, why-how" style** — covering:

1. **Feedforward Neural Network (MLP) Model**
  2. **Preparing Data for Binary Classification**
  3. **Training and Evaluation**
  4. **Salami Attack Detection Function**
  5. **Anomaly Visualization**
- 

## 8. Feedforward Neural Network (MLP) & Data Preparation

### **Model Building and Training Block**

python

CopyEdit

```
Neural network model
```

```
def build_model(input_dim):
```

```
 model = Sequential([
```

```
 Dense(64, activation='relu', input_dim=input_dim),
```

```
 Dense(32, activation='relu'),
```

```
Dense(1, activation='sigmoid') # Output: 1 for anomaly, 0 for normal
])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

return model
```

### What is happening here?

This function builds a basic **Multi-Layer Perceptron (MLP)** for **binary classification** (fraud or not fraud).

### Why use this structure?

- MLPs are **fully connected** neural networks — each neuron in one layer connects to all neurons in the next.
- They are perfect for pattern recognition tasks, like identifying fraudulent patterns in numerical data.
- Simple and effective for non-sequential problems — like checking each transaction individually.

### How it works:

- Sequential(...): Layers are stacked linearly.
  - Dense(64, activation='relu', input\_dim=input\_dim):
    - First hidden layer with 64 neurons and ReLU activation.
    - input\_dim defines the number of features in your data (in this case, 1 feature: Amount).
  - Dense(32, activation='relu'): Second hidden layer.
  - Dense(1, activation='sigmoid'):
    - Output layer with a sigmoid activation (output between 0 and 1).
    - Used for binary classification: 0 = normal, 1 = anomaly (potential fraud).
  - model.compile(...):
    - adam: Optimizer that adapts the learning rate automatically.
    - binary\_crossentropy: Loss function for binary output (fraud/not fraud).
    - metrics=['accuracy']: Monitors how many predictions were correct during training.
-

## Data Preparation for Classification

python

CopyEdit

```
Prepare data for training
```

```
def prepare_data(df):
 scaler = MinMaxScaler()

 features = scaler.fit_transform(df[['Amount']])

 labels = (df['Amount'] % 0.01 != 0).astype(int) # Simulated anomaly labels for salami
 attack

 X_train, X_test, y_train, y_test = train_test_split(features, labels, test_size=0.2,
 random_state=42)

 return X_train, X_test, y_train, y_test, scaler
```

### Why and How:

- Normalizes the data again using MinMaxScaler — just like before.
  - **Label creation logic:**
    - `df['Amount'] % 0.01 != 0`: This checks whether the transaction amount is **not a multiple of 0.01** — a typical indicator of salami fraud (which steals amounts like 0.003, 0.006 etc.)
    - `.astype(int)`: Converts boolean values to integers → True → 1 (fraud), False → 0 (normal).
  - `train_test_split(...)`: Splits data into training (80%) and test (20%) sets.
  - Returns everything for model training.
- 

## Model Training

python

CopyEdit

```
X_train, X_test, y_train, y_test, scaler = prepare_data(transactions_df)

model = build_model(input_dim=X_train.shape[1])

model.fit(X_train, y_train, epochs=20, batch_size=16, validation_data=(X_test, y_test))
```

- `X_train.shape[1]`: Gets the number of features to define the model input layer.
  - `model.fit(...)`: Trains the model using:
    - 20 epochs: 20 passes over the training data.
    - Batch size of 16: Weights are updated after every 16 samples.
    - `validation_data`: Tests the model on new data after each epoch.
- 

## 9. Detecting Salami Attacks in Data

python

CopyEdit

```
def detect_salami_attack(model, df, scaler):
 features = scaler.transform(df[['Amount']])
 predictions = model.predict(features)
 df['Anomaly_Score'] = predictions
```

### Step-by-step:

- Normalizes the input again using the same scaler.
  - Predicts anomaly scores using the trained model.
  - Stores predictions in a new column called 'Anomaly\_Score' — values near 1 mean high chance of fraud.
- 

python

CopyEdit

```
anomalies = df[df['Anomaly_Score'] > 0.5] # Threshold for anomaly detection
```

### What it does:

- Filters all transactions where the model thinks there's over 50% chance it's an anomaly (fraud).
  - You can change this threshold to make the detection more or less sensitive.
- 

python



CopyEdit

```
if anomalies.empty:
 print("No salami attack detected.")
```

else:

```
 print("Salami attack detected!")
```

- Provides a simple message depending on whether anomalies were found.
- 

python

CopyEdit

```
salami_transactions = anomalies[anomalies['Amount'] < 0.1] # Transactions under
$0.10
```

#### **Special filter:**

- Focuses only on very **tiny transactions** — less than 0.1 — which are characteristic of **salami attacks**.
  - This is a heuristic to reduce false positives.
- 

python

CopyEdit

```
if salami_transactions.empty:
 print("No salami attack transactions found.")

else:
 print(f"Number of potential salami attack transactions: {len(salami_transactions)}")
 print("\nPotential salami attack transactions:")
 print(salami_transactions[['Date', 'Description', 'Amount', 'Anomaly_Score']])
 total_amount_in_threat = salami_transactions['Amount'].sum()
 print(f"\nTotal amount in threat: {total_amount_in_threat:.2f}")
```

#### **Report output:**

- If suspected salami transactions are found:

- Prints the **count**, **details**, and **total amount** of these micro-thefts.
  - Very useful for banks to investigate these small fraud patterns.
- 

### Run Detection

python

CopyEdit

```
salami_transactions = detect_salami_attack(model, transactions_df, scaler)
```

- Calls the detection function on the full dataset using the trained model.
- 

### 10. Visualize Detected Anomalies

python

CopyEdit

```
def visualize_anomalies(df, anomalies):
 plt.figure(figsize=(12, 6))
 sns.scatterplot(data=df, x='Date', y='Amount', label='Normal Transactions')
 sns.scatterplot(data=anomalies, x='Date', y='Amount', color='red', label='Anomalies')
 plt.title('Transaction Anomalies')
 plt.xlabel('Date')
 plt.ylabel('Amount')
 plt.legend()
 plt.show()
```

### What it does:

- Plots all transactions (df) and overlays the anomalies (anomalies) in red.
  - Helps visualize **where anomalies occur in time** and **what amounts** are involved.
  - Can clearly reveal **clusters of micro-transactions**, often missed by traditional methods.
- 

python

CopyEdit

visualize\_anomalies(transactions\_df, anomalies)

- Calls the function to show the plot.
- 

### Final Summary

You now have a system that:

1. Accepts PDF/CSV/Excel files with transaction data.
2. Cleans and preprocesses the data.
3. Builds two models:
  - An **LSTM** for sequence prediction.
  - An **MLP (feedforward net)** for anomaly classification.
4. Detects salami attacks based on tiny amounts and pattern deviations.
5. Visualizes the anomalies for better decision-making.